EDITORA OMNIS SCIENTIA

INTEGRIDADE, SEGURANÇA E CONFORMIDADE COM A LGPD EM BANCO DE DADOS: UM GUIA PRÁTICO COM SQL

INFORMAÇÕES GERAIS DO LIVRO TÉCNICO

O número mínimo de capítulos do livro digital (*e-Book*) deve ser de 5 (cinco) capítulos, desde que tenha, no mínimo, 50 páginas. O livro não será comercializado, no intuito de divulgar o conhecimento de forma ampla e gratuita.

Ao livro, será atribuído o *International Standard Book Number* (ISBN), a ficha catalográfica, o código de barras, o *Digital Object Identifier* (DOI), cujo prefixo é da própria editora. Também será licenciado com uma Licença Creative Commons – Atribuição-NãoComercial-SemDerivações 4.0 Internacional. Assim como os serviços de diagramação, design e publicação no site da editora.

Vale ressaltar que o(s) autor(es) ou organizador(es) têm que fazer a apresentação (descrição) do livro, contendo no máximo quatro parágrafos, não excedendo uma página.

Por se tratar de um livro técnico, os autores/organizadores são apresentados exclusivamente no início da obra, e os capítulos desenvolvem livremente os tópicos técnicos, finalizando com as referências.

APRESENTAÇÃO

A evolução dos ecossistemas digitais e a centralidade da informação nas organizações contemporâneas transformaram os Sistemas Gerenciadores de Bancos de Dados (SGBDs) no ativo mais estratégico da infraestrutura tecnológica. Contudo, a simples capacidade de armazenar grandes volumes de registros não é suficiente: a consistência, a blindagem contra acessos indevidos e a rigorosa observância dos marcos regulatórios de privacidade definem o sucesso ou a vulnerabilidade das soluções corporativas.

Este livro técnico foi concebido com uma metodologia estritamente prática, baseada na premissa de que a consolidação de conceitos avançados de engenharia de dados ocorre pela experimentação direta, pela observação de falhas induzidas e pela resolução guiada de cenários do mundo real. Ao longo de cinco capítulos estruturados, o leitor é conduzido desde os mecanismos formais de integridade declarativa e procedural até as arquiteturas modernas de defesa e conformidade normativa em ambiente MySQL.

A obra aborda em profundidade as restrições de domínio, entidade e integridade referencial, o uso estratégico de gatilhos (*triggers*) e procedimentos armazenados, as matrizes de privilégios com papéis (*roles*) sob o princípio do menor privilégio, bem como as técnicas criptográficas simétricas e algoritmos de resumo criptográfico (*hash*). Culmina na implementação direta dos postulados da Lei Geral de Proteção de Dados Pessoais (LGPD – Lei nº 13.709/2018), demonstrando inventário de dados, anonimização, pseudonimização, trilhas de auditoria contínua e atendimento tempestivo aos direitos dos titulares.

Destinado a estudantes de cursos técnicos, graduações em Computação e Engenharia de Software, analistas de sistemas e administradores de bancos de dados (DBAs), este guia serve como referência teórica e laboratório prático para todos que buscam projetar e manter bases de dados robustas, seguras e em plena conformidade com as melhores práticas de governança da informação.

ORGANIZAÇÃO E AUTORIA

Prof. Júlio César
Docente da Faculdade de Engenharia de Software da Universidade de Rio Verde (UniRV) e Instrutor de Informática na Escola SENAI (Unidade Fernando Bezerra).
Atuante no ramo de tecnologia há mais de 10 anos, com expertise nas áreas de dados e inteligência de negócios, projetos de software e requisitos de software.

CAPÍTULO 1
FUNDAMENTOS DE INTEGRIDADE DE DADOS: RESTRIÇÕES DE DOMÍNIO E ENTIDADE EM SGBDs RELACIONAIS

1. CONTEXTUALIZAÇÃO DO TEMA E A IMPORTÂNCIA DO APRENDIZADO

No cenário tecnológico contemporâneo, os dados constituem o ativo mais valioso e estratégico das organizações, fundamentando desde operações rotineiras de faturamento até complexos algoritmos de inteligência artificial e suporte à decisão executiva. Contudo, a utilidade de qualquer base de dados é diretamente proporcional à sua precisão, consistência e confiabilidade. É nesse ponto que reside a importância fundamental de estudar e dominar as **Restrições de Integridade de Dados** em Sistemas Gerenciadores de Bancos de Dados Relacionais (SGBDs).

Compreender e implementar a integridade de dados é uma competência essencial para cientistas da computação, engenheiros de software e administradores de banco de dados (DBAs). A integridade não é apenas uma convenção de nomenclatura ou detalhe estético da modelagem: é o conjunto de regras matemáticas e lógicas impositivas que asseguram que o estado do banco reflita com fidelidade a realidade operacional do negócio.

1.1. A Problemática Crítica da Ausência de Integridade no SGBD

Uma das falhas arquiteturais mais recorrentes no desenvolvimento de software consiste na crença equivocada de que as validações de dados podem ser delegadas exclusivamente às camadas externas, como o formulário no navegador do cliente (*front-end*) ou as rotinas de validação de serviços (*back-end*).

Quando um banco de dados é projetado sem salvaguardas declarativas no próprio esquema, o ecossistema corporativo fica exposto a riscos severos:

- **Contaminação Silenciosa da Base de Dados:** Uma falha de código na aplicação, uma atualização de versão mal sucedida ou um script de migração manual pode inserir registros completamente anômalos (como idades negativas, preços zerados ou e-mails nulos). Uma vez gravado o dado corrompido, sua detecção e saneamento demandam custos operacionais vultosos.

- **Proliferação de Pontos de Falha:** Em arquiteturas distribuídas e ecossistemas de microsserviços, dezenas de aplicações heterogêneas (APIs em Python, serviços em Java, rotinas ETL e painéis de BI) acessam a mesma base de dados. Delegar a validação aos clientes exige que cada desenvolvedor reescreva as mesmas regras em cada linguagem, aumentando exponencialmente a probabilidade de divergências cadastrais.

- **Invalidação de Relatórios Estratégicos e Tomada de Decisão:** Relatórios analíticos e painéis de *Business Intelligence* dependem da higidez dos dados agregados. Se o banco aceita cadastros duplicados ou campos de negócio corrompidos, métricas financeiras e acadêmicas tornam-se espúrias, induzindo a liderança corporativa a decisões desastrosas.

- **Descumprimento Regulatório e Vulnerabilidade Jurídica:** Em setores regulados (educação, saúde e finanças), o registro de dados incoerentes sujeita a instituição a penalidades administrativas, autuações fiscais e sanções perante órgãos fiscalizadores.

O princípio fundamental da engenharia de dados dita que o SGBD deve agir como a **última e inegociável linha de defesa** da informação corporativa. Mesmo que uma requisição chegue corrompida por qualquer motivo, o motor relacional deve rejeitá-la prontamente antes de sua consolidação em disco.

2. FUNDAMENTAÇÃO TEÓRICA: O MODELO RELACIONAL E A TAXONOMIA DE INTEGRIDADE

A formulação teórica da integridade de dados remonta à criação do próprio modelo relacional por Edgar F. Codd (1970). Em seu trabalho seminal para a ACM, Codd postulou que a independência física e lógica dos dados exige que a consistência semântica seja mantida de forma impositiva pela estrutura de dados do banco, e não por procedimentos espalhados em programas de aplicação.

A literatura clássica de bancos de dados (DATE, 2004; ELMASRI; NAVATHE, 2011; SILBERSCHATZ; KORTH; SUDARSHAN, 2020) classifica a integridade em quatro categorias basilares:

1. **Integridade de Domínio:** Assegura que todo valor armazenado em um atributo pertença ao domínio de valores válidos e autorizados para aquele campo (tipo de dado primitivo, tamanho, intervalo aceitável, padrão de máscara textual e restrição contra valores nulos).

2. **Integridade de Entidade:** Estabelece que nenhuma relação contenha tuplas indistinguíveis. Exige formalmente a existência de uma chave primária (*Primary Key*) cujos valores componentes não admitam repetição e jamais assumam estado nulo (*null*).

3. **Integridade Referencial:** Rege as associações inter-relacionais por meio de chaves estrangeiras (*Foreign Keys*), garantindo que referências cruzadas apontem invariavelmente para registros mestres existentes.

4. **Integridade Definida pelo Usuário:** Abrange restrições de negócio complexas da organização que exigem validações procedurais dinâmicas (frequentemente implementadas via gatilhos ou procedimentos armazenados).

Neste capítulo, aprofundaremos as duas primeiras categorias: integridade de domínio e de entidade, confrontando esquemas desprovidos de regras com arquiteturas devidamente blindadas no MySQL 8.0+.

3. O ESTUDO DE CASO PRÁTICO: O CENÁRIO SEM REGRAS

Para compreender a mecânica e a gravidade dos problemas causados pela omissão de restrições no projeto do esquema, examinaremos o comportamento de uma tabela concebida de forma deliberadamente permissiva.

```
DROP DATABASE IF EXISTS escola;
CREATE DATABASE escola CHARACTER SET utf8mb4;
USE escola;

-- Esquema vulneravel: ausente de chaves e restricoes de dominio
CREATE TABLE sem_regras (
    id INT,
    idade INT,
    email VARCHAR(100)
);
```

Executamos a inserção de transações que representam falhas típicas de integração de formulários web:

```
INSERT INTO sem_regras VALUES
(1, -5, NULL),
(1, 200, 'x');

SELECT * FROM sem_regras;
```

3.1. Análise Técnica das Anomalias Geradas

- **Ambiguidade de Entidade:** O código identificador `id = 1` foi registrado duas vezes. É impossível determinar matematicamente qual linha corresponde a qual aluno no mundo real.
- **Incoerência de Domínio Numérico:** O atributo `idade` aceitou simultaneamente o valor negativo (-5), biologicamente impossível, e o valor 200, que extrapola o limite fisiológico humano.
- **Ausência e Degradação de Formato:** O atributo `email` aceitou a ausência total de conteúdo (`NULL`) e uma cadeia monossilábica (`'x'`), que quebra rotinas automáticas de despacho de mensagens.

O banco comportou-se como mero gravador passivo de bytes. É exatamente para sanar essa vulnerabilidade estrutural que aplicamos a integridade de domínio.

4. RESTRIÇÕES DE INTEGRIDADE DE DOMÍNIO EM SQL

As restrições de domínio delimitam o espaço de estados válidos de uma coluna por meio das cláusulas `CHECK`, `NOT NULL` e `DEFAULT` (MELTON; SIMON, 2002).

4.1. Requisitos Técnicos no MySQL: O Suporte Efetivo ao CHECK

É imperativo destacar uma particularidade técnica do SGBD MySQL: até a versão 8.0.15, o analisador sintático reconhecia a palavra reservada `CHECK`, mas não aplicava a validação durante as operações de escrita. O suporte formal e executável à cláusula `CHECK` foi introduzido a partir da versão **MySQL 8.0.16** (ORACLE, 2024).

A confirmação da versão operacional é verificada por:

```
SELECT VERSION();
```

4.2. Implementação das Quatro Salvaguardas de Domínio

Refatoramos a relação cadastral aplicando um conjunto coordenado de restrições:

1. **Intervalo Numérico com `CHECK`:** Impõe que a idade pertença obrigatoriamente à faixa etária escolar entre 14 e 99 anos (`idade BETWEEN 14 AND 99`).
2. **Obrigatoriedade com `NOT NULL`:** Veda a criação de tuplas sem endereço de e-mail.
3. **Carga Padrão com `DEFAULT`:** Preenche automaticamente o estado cadastral como `'ATIVO'` quando o parâmetro não for informado na instrução DML.
4. **Máscara Estrutural com `LIKE`:** Restringe a matrícula acadêmica para que se inicie compulsoriamente com o padrão do curso (`'BD2%'`).

```
CREATE TABLE aluno (
    idade INT CHECK (idade BETWEEN 14 AND 99),
    email VARCHAR(100) NOT NULL,
    status VARCHAR(10) DEFAULT 'ATIVO',
    matricula CHAR(10) CHECK (matricula LIKE 'BD2%')
);
```

4.3. Testes Práticos e Análise de Falhas Provocadas

A validação em laboratório é realizada executando transações válidas e forçando os limites do motor com desvios intencionais:

```
-- CASO 1: Registro em plena conformidade com as restricoes
INSERT INTO aluno (idade, email, matricula)
VALUES (17, 'ana@escola.com', 'BD2-001');

-- CASO 2: Violacao de intervalo de idade (idade = 200)
-- Retorno do motor: Error 3819 (HY000): Check constraint 'aluno_chk_1' is violated.
INSERT INTO aluno (idade, email, matricula)
VALUES (200, 'x@x.com', 'BD2-002');

-- CASO 3: Violacao de obrigatoriedade (e-mail nulo)
-- Retorno do motor: Error 1048 (23000): Column 'email' cannot be null.
INSERT INTO aluno (idade, email, matricula)
VALUES (17, NULL, 'BD2-003');

-- CASO 4: Violacao de padrao textual (matricula com prefixo incorreto)
-- Retorno do motor: Error 3819 (HY000): Check constraint 'aluno_chk_2' is violated.
INSERT INTO aluno (idade, email, matricula)
VALUES (17, 'y@y.com', 'XYZ-004');
```

Ao inspecionar o estado resultante:

```
SELECT * FROM aluno;
```

Constata-se que apenas a tupla da aluna Ana foi gravada. O SGBD interceptou e abortou cada tentativa anômala, blindando o repositório contra dados corrompidos.

4.4. Evolução do Esquema com ALTER TABLE

Em sistemas corporativos em produção, a introdução de novas regras em tabelas pré-existentes é realizada com o comando `ALTER TABLE ... ADD CONSTRAINT`:

```
-- Adicao de verificacao sintatica basica do caractere '@' no e-mail
ALTER TABLE aluno ADD CONSTRAINT chk_email
CHECK (email LIKE '%@%');

-- Teste de violacao da regra recem-criada
```

```
6  -- Retorno esperado: Error 3819 referente a 'chk_email'
7  INSERT INTO aluno (idade, email, matricula)
8  VALUES (17, 'semarroba', 'BD2-005');
9
10 DROP TABLE aluno;
```

5. RESTRIÇÕES DE INTEGRIDADE DE ENTIDADE

A integridade de entidade declara que cada elemento registrado na relação deve possuir uma identidade individualizada e imutável. No modelo relacional, essa imposição é cumprida pela definição da **Chave Primária** (*Primary Key* – PK), acompanhada de restrições de unicidade (`UNIQUE`) para chaves naturais de negócio (DATE, 2004).

5.1. Chaves Substitutas (Surrogate Keys) versus Chaves Naturais

A melhor prática de arquitetura em bancos de dados corporativos preconiza o desacoplamento entre a chave física do registro e os atributos identificadores do negócio:

- **Chave Substituta (*Surrogate Key*):** Um identificador numérico autoincrementado (`AUTO_INCREMENT`), controlado estritamente pelo motor do banco, desprovido de significado semântico e imutável ao longo do ciclo de vida da tupla.
- **Chave Natural Protegida:** Um documento de identificação civil (como o CPF), protegido pela cláusula `UNIQUE NOT NULL`. Essa dissociação impede que correções cadastrais na chave natural afetem tabelas dependentes ligadas por chaves estrangeiras.

```sql
CREATE TABLE aluno (
    id_aluno INT PRIMARY KEY AUTO_INCREMENT,
    cpf CHAR(11) UNIQUE NOT NULL,
    nome VARCHAR(80) NOT NULL,
    idade INT CHECK (idade BETWEEN 14 AND 99),
    email VARCHAR(100) NOT NULL,
    status VARCHAR(10) DEFAULT 'ATIVO'
);
```

5.2. Testando a Barreira de Unicidade (UNIQUE)

Inserimos duas alunas distintas e forçamos a inserção de uma terceira tupla duplicando o CPF da primeira:

```sql
INSERT INTO aluno (cpf, nome, idade, email) VALUES
('11111111111', 'Ana', 16, 'a@e.com'),
('22222222222', 'Bia', 17, 'b@e.com');

```

```
-- Tentativa de clonagem/duplicacao de CPF
-- Retorno: Error 1062 (23000): Duplicate entry '11111111111' for key 'aluno.
    cpf'
INSERT INTO aluno (cpf, nome, idade, email) VALUES
('11111111111', 'Caio', 18, 'c@e.com');
```

Da mesma forma, uma tentativa de sobregravar manualmente uma chave primária existente é sumariamente repelida:

```
-- Tentativa de sobrescrever o id_aluno 1 ja existente
-- Retorno: Error 1062 (23000): Duplicate entry '1' for key 'aluno.PRIMARY'
INSERT INTO aluno (id_aluno, cpf, nome, idade, email)
VALUES (1, '33333333333', 'Caio', 18, 'c@e.com');
```

5.3. Chaves Primárias Compostas

Em relações associativas que conectam entidades distintas em cardinalidade muitos-para-muitos ($N : M$), a chave primária é formada pela tupla combinada das chaves de associação:

```
CREATE TABLE turma (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(50) NOT NULL
);

-- Tabela associativa com Chave Primaria Composta
CREATE TABLE matricula (
    id_aluno INT,
    id_turma INT,
    PRIMARY KEY (id_aluno, id_turma)
);

INSERT INTO turma (nome) VALUES ('Info 2A'), ('Info 2B');

-- Insercoes validas
INSERT INTO matricula VALUES (1, 1), (2, 1);

-- Mesmo aluno em turma diferente: OPERACAO PERMITIDA
INSERT INTO matricula VALUES (1, 2);

-- Tentativa de duplicar o mesmo aluno na mesma turma: BLOQUEIO IMEDIATO
-- Retorno: Error 1062: Duplicate entry '1-1' for key 'matricula.PRIMARY'
```

```
23 INSERT INTO matricula VALUES (1, 1);
```

6. LABORATÓRIO PRÁTICO ORIENTADO

6.1. Exercício 1: Modelagem e Restrições na Tabela Produto

Problemática do Caso: O departamento de estoque de uma instituição comercial necessita de uma tabela para cadastramento de produtos. Caso preços negativos ou nomes vazios sejam aceitos, sistemas de frente de caixa travam ou computam perdas financeiras. Projete a tabela aplicando chave primária autoincrementada, nome obrigatório, preço obrigatoriamente superior a zero e estoque com valor padrão zero.

```sql
CREATE TABLE produto (
    id_produto INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL,
    preco DECIMAL(10,2) CHECK (preco > 0),
    estoque INT DEFAULT 0
);

-- Insercao valida
INSERT INTO produto (nome, preco) VALUES ('Apostila de Banco de Dados', 45.00);

-- Erro provocado 1: Preco zero violando o CHECK (Erro 3819)
INSERT INTO produto (nome, preco) VALUES ('Caneta', 0.00);

-- Erro provocado 2: Nome ausente violando o NOT NULL (Erro 1048)
INSERT INTO produto (nome, preco) VALUES (NULL, 19.90);
```

6.2. Exercício 2: Regra de Negócio Salarial

Problemática do Caso: No cadastro funcional corporativo, o departamento jurídico exige conformidade estrita com o piso salarial nacional estabelecido em R$ 1.518,00. Qualquer inserção abaixo desse limite representa risco de passivo trabalhista e deve ser impedida pelo SGBD.

```sql
CREATE TABLE funcionario (
    id_func INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL,
    salario DECIMAL(10,2) CHECK (salario >= 1518.00)
);

-- Transacao homologada
INSERT INTO funcionario (nome, salario) VALUES ('Carla Mendes', 2800.00);
```

```
9
10 -- Rejeicao imperativa pelo SGBD (abaixo do piso: Erro 3819)
11 INSERT INTO funcionario (nome, salario) VALUES ('Davi Alcantara', 1100.00);
```

7. SÍNTESE DO CAPÍTULO E QUADRO COMPARATIVO

Categoria	Cláusula	Finalidade Arquitetural	Código de Erro
Domínio	`CHECK`	Validação de limites numéricos ou padrões textuais	Erro 3819
Domínio	`NOT NULL`	Obrigatoriedade compulsória de preenchimento	Erro 1048
Domínio	`DEFAULT`	Carga de valor padrão na ausência de input	–
Entidade	`PRIMARY KEY`	Identificação única, absoluta e imutável da tupla	Erro 1062
Entidade	`UNIQUE`	Garantia de unicidade para chaves naturais de negócio	Erro 1062

Tabela 1: Mecanismos de integridade de domínio e entidade no MySQL.

REFERÊNCIAS DO CAPÍTULO

CODD, E. F. A relational model of data for large shared data banks. **Communications of the ACM**, v. 13, n. 6, p. 377-387, 1970.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

MELTON, J.; SIMON, A. R. **SQL: 1999 - Understanding Relational Language Components**. San Francisco: Morgan Kaufmann, 2002.

ORACLE. **MySQL 8.0 Reference Manual: Constraints and DDL**. Redwood City: Oracle Corporation, 2024. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 13 out. 2026.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

CAPÍTULO 2
INTEGRIDADE REFERENCIAL E REGRAS ATIVAS DE NEGÓCIO COM GATILHOS (TRIGGERS)

1. CONTEXTUALIZAÇÃO DO TEMA E A IMPORTÂNCIA DO APRENDIZADO

A arquitetura de um Sistema Gerenciador de Banco de Dados Relacional atinge sua maturidade quando as tabelas deixam de ser ilhas isoladas de informação e passam a refletir a complexa rede de relacionamentos existente no mundo real. Enquanto as restrições de domínio e entidade apresentadas no Capítulo 1 blindam colunas e tuplas individuais, a **Integridade Referencial** e a **Automação Procedural via Gatilhos (Triggers)** constituem a espinha dorsal da consistência entre relações interconectadas.

Dominar esses dois pilares é indispensável para o profissional de tecnologia que atua com arquitetura de software, engenharia de dados e infraestrutura corporativa. No ecossistema empresarial, a perda de integridade entre dados associados acarreta prejuízos financeiros diretos, paradas operacionais e severas distorções contábeis.

1.1. A Problemática Crítica da Ausência de Integridade Referencial

A negligência na implementação formal de chaves estrangeiras e regras ativas introduz patologias graves na base de dados:

- **Proliferação de Registros Órfãos:** Quando tabelas dependentes (como itens de venda, parcelas de financiamento ou matrículas acadêmicas) não são formalmente atreladas a uma chave estrangeira, a exclusão acidental ou intencional de um registro pai (como um cliente ou um curso) deixa tuplas filhas desconectadas. O sistema passa a conter vendas faturadas para clientes que não existem, quebrando conciliações financeiras e a emissão de notas fiscais.

- **Inconsistências em Consultas com Junção (*JOIN*):** Registros órfãos ou chaves desincronizadas corrompem relatórios gerenciais construídos com `INNER JOIN`, que passam a omitir dados silenciosamente, distorcendo o faturamento reportado à diretoria e a órgãos governamentais.

- **Destruição em Cascata Descontrolada:** A adoção inadvertida ou ausente de po-

líticas de deleção pode levar a catástrofes: ou o banco impede operações rotineiras sem justificativa clara, ou propaga exclusões em massa (`CASCADE`) que deletam históricos contratuais vitais por engano.

1.2. A Necessidade de Lógica Ativa: As Limitações das Restrições Estáticas

Outra problemática fundamental reside no fato de que regras de negócio complexas não podem ser resolvidas apenas com declarações primitivas de chave. Considere validações condicionais dependentes de múltiplos atributos ou que exigem verificações cronológicas (por exemplo: impedir que uma devolução de material ocorra antes da data da sua retirada, ou bloquear saques que ultrapassem um limite diário flutuante).

No padrão ANSI SQL teórico, tais validações gerais deveriam ser suportadas por *ASSERTIONS* globais de esquema (MELTON; SIMON, 2002). No entanto, devido à altíssima complexidade computacional para verificar assertivas a cada instrução de escrita, a imensa maioria dos SGBDs de mercado (incluindo o MySQL) não implementa *ASSERTIONS*. Nesse contexto, o domínio de **Gatilhos** (*Triggers*) torna-se mandatório: eles representam o mecanismo procedural padrão da indústria para programar a integridade definida pelo usuário e fazer o banco de dados reagir ativamente a violações antes que dados corrompidos sejam gravados.

2. FUNDAMENTAÇÃO TEÓRICA DA INTEGRIDADE REFERENCIAL

A integridade referencial fundamenta-se no princípio formal de que toda referência cruzada entre duas relações no modelo relacional deve ser plenamente válida e comprovável (CODD, 1970; DATE, 2004).

Conforme formalizado por Elmasri e Navathe (2011), uma restrição de integridade referencial é especificada entre duas relações: a **relação referenciadora** (R_1, ou tabela filha) e a **relação referenciada** (R_2, ou tabela pai). Uma tupla t_1 em R_1 que possui um conjunto de atributos de chave estrangeira FK deve satisfazer à condição de que, se o valor de FK não for nulo, esse valor deve coincidir rigorosamente com a chave primária PK de alguma tupla t_2 em R_2:

$$t_1[FK] = t_2[PK] \quad \text{ou} \quad t_1[FK] = \text{NULL} \tag{1}$$

Quando esse predicado é garantido pelo SGBD, o sistema torna-se matematicamente imune ao surgimento de registros órfãos.

3. MECÂNICA DE CHAVES ESTRANGEIRAS E POLÍTICAS REFERENCIAIS

A declaração de uma chave estrangeira estabelece o vínculo formal de integridade entre duas tabelas no MySQL.

3.1. Políticas de Ação no Descarte e Atualização

O arquiteto de banco de dados deve prever o que o motor relacional fará caso uma tupla na tabela pai sofra tentativa de exclusão ou alteração. O SQL oferece três políticas fundamentais:

1. **`ON DELETE RESTRICT / NO ACTION` (Padrão):** Veda e cancela a operação no pai enquanto existir qualquer tupla filha associada. É a salvaguarda primária contra deleção acidental de registros mestres.

2. **`ON DELETE CASCADE`:** Remove compulsoriamente todas as tuplas filhas quando a tupla pai for excluída. Aplica-se a relações de dependência existencial estrita (como itens de uma fatura ou tabelas de mapeamento técnico).

3. **`ON DELETE SET NULL`:** Desconecta as tuplas filhas da tupla pai, atribuindo `NULL` à chave estrangeira (exige que a coluna permita nulos).

3.2. Implementação e Análise de Falha Provocada

Examinamos a implementação entre a tabela mestre de cursos e a tabela dependente de matrículas:

```
USE escola;

-- Tabela Pai (Referenciada)
CREATE TABLE curso (
    id_curso INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(60) NOT NULL
);

-- Tabela Filha com Chave Estrangeira e Politica RESTRICT
CREATE TABLE matricula_curso (
    id_matricula INT PRIMARY KEY AUTO_INCREMENT,
    id_curso INT NOT NULL,
    aluno_nome VARCHAR(80) NOT NULL,
```

```
    FOREIGN KEY (id_curso) REFERENCES curso(id_curso)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

INSERT INTO curso (nome) VALUES ('Engenharia de Software');
INSERT INTO matricula_curso (id_curso, aluno_nome) VALUES (1, 'Marcos Silva');

-- Tentativa de exclusao do Curso 1 enquanto existirem matriculas ativas
-- Retorno do SGBD: Error 1451 (23000): Cannot delete or update a parent row: a
     foreign key constraint fails
DELETE FROM curso WHERE id_curso = 1;
```

O SGBD cancelou a operação com o Erro 1451, impedindo que o aluno Marcos Silva ficasse órfão no sistema institucional.

4. ARQUITETURA E PROGRAMAÇÃO DE GATILHOS (TRIGGERS)

Um **Gatilho** (*Trigger*) em SQL é um bloco de código procedural autônomo, vinculado a uma relação de dados, cuja execução é disparada compulsoriamente pelo motor transacional do SGBD diante de uma instrução DML (`INSERT`, `UPDATE` ou `DELETE`) (MELTON; SIMON, 2002; SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

4.1. Os Quatro Pilares Estruturais de um Gatilho

Pilar	Papel no Ciclo Transacional
1. Tabela Alvo	Relação na qual as mutações são monitoradas pelo motor.
2. Evento Disparador	Operação modificadora: `INSERT`, `UPDATE` ou `DELETE`.
3. Momento de Disparo	• `BEFORE`: Executado antes da escrita no log transacional. Essencial para validação, sanitização e cancelamento prévio de operações inválidas. • `AFTER`: Executado após a gravação da tupla. Ideal para auditoria contínua, replicação e logs históricos.
4. Granularidade	• `FOR EACH ROW`: Disparado individualmente para cada tupla processada. • `FOR EACH STATEMENT`: Disparado uma única vez por comando.

Tabela 2: Os quatro pilares fundamentais da arquitetura de um gatilho.

4.2. As Variáveis de Contexto NEW e OLD

Em gatilhos linha a linha (`FOR EACH ROW`), o motor do banco expõe variáveis contextuais em memória:

- `NEW`: Permite inspecionar e intervir sobre os novos valores propostos para a tupla (disponível em operações de `INSERT` e `UPDATE`).
- `OLD`: Permite inspecionar o estado prévio dos atributos antes da modificação (disponível em operações de `UPDATE` e `DELETE`).

4.3. Delimitadores e Interrupção Transacional com SIGNAL

Para que blocos procedurais contenham múltiplos comandos terminados em ponto e vírgula, altera-se temporariamente o delimitador da sessão com `DELIMITER $$`.

Para abortar a transação e notificar a aplicação sobre uma violação de regra de negócio, o gatilho executa `SIGNAL SQLSTATE '45000'` com a mensagem descritiva

(ORACLE, 2024). A classe `'45000'` é reservada pelo padrão ANSI para representar exceções definidas pelo usuário.

5. ESTUDO DE CASO PRÁTICO: VALIDAÇÃO TEMPORAL DE EMPRÉSTIMOS

Para ilustrar uma regra de negócio condicional impossível de ser modelada por chaves estáticas simples, examinamos o subsistema de empréstimos.

5.1. Modelagem da Tabela

```sql
CREATE TABLE emprestimo (
    id INT PRIMARY KEY AUTO_INCREMENT,
    livro VARCHAR(80) NOT NULL,
    data_retirada DATE NOT NULL,
    data_devolucao DATE
);
```

Regra de Negócio Formal: Uma devolução jamais pode possuir data cronologicamente anterior à data de retirada (`data_devolucao >= data_retirada`).

5.2. Construção do Gatilho BEFORE INSERT

```sql
DELIMITER $$

CREATE TRIGGER valida_devolucao
BEFORE INSERT ON emprestimo
FOR EACH ROW
BEGIN
    -- Validacao condicional comparando os campos do registro NEW proposto
    IF NEW.data_devolucao IS NOT NULL AND NEW.data_devolucao < NEW.
        data_retirada THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Data invalida: devolucao anterior a retirada.';
    END IF;
END$$

DELIMITER ;
```

5.3. Testes de Laboratório e Análise do Retorno

```sql
-- CASO 1: Transacao cronologicamente valida
INSERT INTO emprestimo VALUES
```

```
(NULL, 'Dom Casmurro', '2026-09-01', '2026-09-15');

-- CASO 2: Anomalia cronologica intencional (devolucao antes da retirada)
-- Retorno do motor: Error 1644 (45000): Data invalida: devolucao anterior a
    retirada.
INSERT INTO emprestimo VALUES
(NULL, 'Quincas Borba', '2026-09-10', '2026-09-01');
```

O gatilho operou no estágio `BEFORE`, impedindo que a incoerência temporal chegasse ao armazenamento físico.

6. DESAFIO PRÁTICO: A REFATORAÇÃO DA TABELA PEDIDO

Para consolidar as quatro dimensões de integridade estudadas (Domínio, Entidade, Referencial e Definida pelo Usuário), abordamos a refatoração do esquema de pedidos de comércio eletrônico.

6.1. O Ponto de Partida Desprotegido

```sql
CREATE TABLE cliente (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL
);

-- Tabela fragilizada sem regras formais
CREATE TABLE pedido_sem_regras (
    id INT,
    id_cliente INT,
    data_pedido DATE,
    valor DECIMAL(10,2),
    status VARCHAR(15)
);
```

6.2. Resolução do Desafio: Refatoração com Quatro Salvaguardas

```sql
CREATE TABLE pedido (
    id_pedido INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente INT NOT NULL,
    data_pedido DATE NOT NULL,
    valor DECIMAL(10,2) CHECK (valor > 0),
    status VARCHAR(15) DEFAULT 'ABERTO',
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
    ON DELETE RESTRICT ON UPDATE CASCADE
);
```

6.3. Bateria de Testes do Esquema Refatorado

```sql
INSERT INTO cliente (nome) VALUES ('Central de Suprimentos UniRV');

-- Insercao valida (assume DEFAULT 'ABERTO' e cumpre chave estrangeira)
INSERT INTO pedido (id_cliente, data_pedido, valor)
```

```
VALUES (1, '2026-10-15', 420.00);

-- TESTE 1: Cliente inexistente (id = 999)
-- Retorno do motor: Error 1452 (Foreign key constraint fails)
INSERT INTO pedido (id_cliente, data_pedido, valor)
VALUES (999, '2026-10-15', 100.00);

-- TESTE 2: Valor negativo ou nulo
-- Retorno do motor: Error 3819 (Check constraint 'pedido_chk_1' is violated)
INSERT INTO pedido (id_cliente, data_pedido, valor)
VALUES (1, '2026-10-15', -50.00);
```

7. SÍNTESE DO CAPÍTULO E QUADRO CONSOLIDADO

Restrição	Mecanismo SQL	Natureza	Falha Operacional Evitada
Domínio	`CHECK, NOT NULL`	Declarativa	Preços negativos, valores em branco
Entidade	`PK, UNIQUE`	Declarativa	Duplicidade cadastral, ambiguidade
Referencial	`FOREIGN KEY, RESTRICT`	Declarativa	Vendas para clientes excluídos, órfãos
Usuário	`TRIGGER, SIGNAL`	Procedural	Incoerência temporal, regras complexas

Tabela 3: Matriz consolidada de restrições de integridade no SGBD.

REFERÊNCIAS DO CAPÍTULO

CODD, E. F. A relational model of data for large shared data banks. **Communications of the ACM**, v. 13, n. 6, p. 377-387, 1970.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

MELTON, J.; SIMON, A. R. **SQL: 1999 - Understanding Relational Language Components**. San Francisco: Morgan Kaufmann, 2002.

ORACLE. **MySQL 8.0 Reference Manual: Triggers and Stored Programs**. Redwood City: Oracle Corporation, 2024. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 13 out. 2026.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

CAPÍTULO 3
SEGURANÇA EM BANCO DE DADOS: CONTROLE DE ACESSO, MATRIZES DE PRIVILÉGIOS E RBAC

1. CONTEXTUALIZAÇÃO DO TEMA E A IMPORTÂNCIA DO APRENDIZADO

Nos ecossistemas corporativos contemporâneos, o banco de dados representa o cofre digital da organização, concentrando registros financeiros, estratégias de mercado, segredos industriais e milhões de dados pessoais de clientes e cidadãos. Consequentemente, os SGBDs são o alvo prioritário de agentes maliciosos, ataques cibernéticos patrocinados e fraudes internas. Nesse cenário, o domínio formal da **Segurança em Bancos de Dados** e dos **Modelos de Controle de Acesso** transcende a mera rotina operacional: é um imperativo de sobrevivência e conformidade institucional.

Estudar segurança em nível de banco de dados é indispensável porque a segurança perimetral (como firewalls de rede e criptografia de canal) não protege a organização contra ameaças originadas em credenciais comprometidas, falhas em códigos de aplicação ou acessos indevidos por colaboradores com privilégios excessivos.

1.1. A Problemática Crítica do Controle de Acesso Inadequado

A negligência na segregação de acessos e permissões no SGBD desencadeia vulnerabilidades estruturais recorrentes no mercado de trabalho:

- **O Vício Corporativo da Conta de Superusuário Compartilhada:** Uma das más práticas mais nocivas da indústria é conectar aplicações web, microsserviços, desenvolvedores e estagiários utilizando a mesma credencial de superusuário (`root` ou `sa`). Sob esse arranjo, qualquer falha em uma tela da aplicação concede controle irrestrito sobre todo o banco de dados, permitindo a exclusão de esquemas inteiros e a visualização de dados sigilosos.

- **Ataques por Injeção de SQL (*SQL Injection*):** Listado consistentemente entre os riscos mais críticos do guia internacional *OWASP Top 10* (OWASP, 2021), o ataque de injeção permite que atacantes externos subvertam consultas e burlem telas de login quando entradas de usuários são concatenadas diretamente em código SQL sem parametrização.

- **Violação do Princípio do Menor Privilégio:** Quando não há segregação de papéis, funcionários do setor de atendimento podem acessar salários de executivos, professores podem alterar tabelas de configuração ou alunos podem visualizar notas de terceiros. A falta de granularidade por coluna e por tabela expõe a instituição a fraudes e processos judiciais.

- **Proliferação e Descontrole de Permissões:** Em ambientes que não utilizam o modelo de papéis (*Role-Based Access Control* – RBAC), permissões são concedidas individualmente a cada funcionário. Quando um colaborador muda de setor ou é desligado da empresa, suas permissões antigas permanecem ativas na base de dados por anos a fio, criando portas dos fundos desmonitoradas.

O aprendizado deste capítulo instrumentaliza o leitor a implementar o célebre **Princípio do Menor Privilégio** (*Principle of Least Privilege*), assegurando que cada usuário, aplicação ou serviço receba estritamente as permissões mínimas e necessárias para desempenhar sua função funcional, e nada além disso.

2. FUNDAMENTAÇÃO TEÓRICA DA SEGURANÇA EM SGBDs

Os alicerces formais da segurança computacional aplicada a sistemas de informação foram estabelecidos por Saltzer e Schroeder (1975) em seu influente trabalho sobre proteção de informações em computadores. Dentre os princípios formulados, destacam-se a economia de mecanismos, a separação de privilégios e a exigência de que o acesso padrão a qualquer recurso seja rigorosamente nulo até que uma concessão explícita seja homologada.

A literatura moderna de segurança da informação (STALLINGS; BROWN, 2018; SILBERSCHATZ; KORTH; SUDARSHAN, 2020) decompõe a segurança no SGBD em duas barreiras distintas e complementares:

1. **Autenticação:** O mecanismo pelo qual o sistema verifica a veracidade da identidade alegada por um agente externo (“Quem é você?”). Em bancos de dados relacionais, isso é realizado pela combinação de usuário, máquina/host de origem (`'usuario'@'host'`) e segredos criptográficos ou certificados.

2. **Autorização:** O procedimento contínuo pelo qual o motor do SGBD consulta o catálogo de privilégios para determinar se uma identidade autenticada possui o direito legítimo de executar uma instrução DDL ou DML sobre um determinado objeto (“O que você tem permissão de executar?”).

Para estruturar a autorização, a ciência da computação desenvolveu três modelos clássicos de controle de acesso (SANDHU et al., 1996; ELMASRI; NAVATHE, 2011):

- **DAC (*Discretionary Access Control*):** Controle discricionário, onde o criador/dono do objeto decide quem recebe privilégios.

- **MAC (*Mandatory Access Control*):** Controle obrigatório, baseado em políticas impositivas e níveis de classificação de segurança.

- **RBAC (*Role-Based Access Control*):** Controle baseado em papéis organizacionais, o padrão ouro das arquiteturas corporativas modernas.

3. INJEÇÃO DE SQL (SQL INJECTION): ANATOMIA E DEFESA

O ataque de **Injeção de SQL** ocorre quando entradas textuais fornecidas por usuários são concatenadas de forma direta em instruções SQL dinâmicas sem sanitização ou parametrização, permitindo que a entrada do atacante altere a sintaxe e a lógica da consulta planejada pelo desenvolvedor (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

3.1. Demonstração Prática da Vulnerabilidade

Considere uma relação de credenciais de usuários, na qual as senhas são armazenadas como resumo criptográfico (hash SHA-256):

```
USE escola;

CREATE TABLE usuario (
    id INT PRIMARY KEY AUTO_INCREMENT,
    login VARCHAR(30) UNIQUE NOT NULL,
    senha_hash CHAR(64) NOT NULL
);

-- Insercao com hash criptografico SHA-256
INSERT INTO usuario (login, senha_hash) VALUES
('ana', SHA2('ana#123', 256)),
('admin', SHA2('Adm!n#2026', 256));
```

Se a aplicação web autenticar os usuários concatenando strings da seguinte forma:

```
SELECT * FROM usuario WHERE login = 'ana';
```

Um invasor pode submeter no campo de login a carga útil (*payload*) clássica: `' OR '1'='1`. A consulta processada pelo motor do SGBD resultará em:

```
SELECT * FROM usuario WHERE login = '' OR '1'='1';
```

Como a condição booleana `'1'='1'` é invariavelmente verdadeira (*tautologia*), o SGBD retorna todas as tuplas da tabela, concedendo acesso com os privilégios do primeiro usuário listado (comumente o administrador do sistema).

3.2. A Defesa Efetiva: Consultas Preparadas (Prepared Statements)

A única mitigação definitiva contra a injeção de SQL consiste na dissociação estrita entre o plano de compilação do código executável e os parâmetros de dados. No MySQL, isso

é garantido pelo uso de **Comandos Preparados** (*Prepared Statements*):

```
-- 1. Compilacao previa da instrucao SQL com marcador de parametro '?'
PREPARE stmt FROM 'SELECT id, login FROM usuario WHERE login = ?';

-- 2. Teste do payload malicioso passado como dado puro
SET @login_injetado = "' OR '1'='1";
EXECUTE stmt USING @login_injetado;
-- O motor busca literalmente um usuario cujo login seja "' OR '1'='1" e
    retorna zero tuplas!

-- 3. Execucao legitima com parametro autentico
SET @login_valido = 'ana';
EXECUTE stmt USING @login_valido;

-- 4. Desalocacao dos recursos no servidor
DEALLOCATE PREPARE stmt;
```

Com a parametrização, os dados inseridos jamais são interpretados como operadores lógicos pelo analisador sintático do banco de dados.

4. GESTÃO DE CONTAS E POLÍTICAS DE SENHAS NO MYSQL

No MySQL, a identidade de uma conta é composta pelo nome do usuário e pelo computador/rede de origem da conexão (`'usuario'@'host'`). Isso permite isolar conexões locais (`'localhost'`) de acessos remotos via rede corporativa (ex.: `'192.168.1.%'`).

4.1. Criação de Contas e Restrições de Conexão

```
-- Criacao de conta restrita ao host local
CREATE USER 'joao'@'localhost' IDENTIFIED BY 'SenhaForte#2025';

-- Consulta ao catalogo de sistema de usuarios
SELECT user, host, plugin FROM mysql.user WHERE user = 'joao';
```

4.2. Políticas Corporativas de Senhas e Bloqueio por Força Bruta

Para atender a padrões corporativos rigorosos (como normas ISO/IEC 27001 e guias CIS Benchmark), o DBA deve impor políticas de ciclo de vida e proteção contra tentativas repetidas de invasão:

```
-- 1. Expiracao periodica obrigatoria a cada 90 dias
ALTER USER 'joao'@'localhost' PASSWORD EXPIRE INTERVAL 90 DAY;

-- 2. Bloqueio temporario da conta apos 3 tentativas consecutivas incorretas
ALTER USER 'joao'@'localhost'
FAILED_LOGIN_ATTEMPTS 3
PASSWORD_LOCK_TIME 1;
```

4.3. A Separação entre Autenticar e Autorizar

Quando o usuário João estabelece sua conexão:

```
SELECT CURRENT_USER(); -- Retorna 'joao@localhost' (Autenticado com sucesso)

-- Tentativa de consulta na tabela de alunos
-- Retorno do motor: Error 1142 (42000): SELECT command denied to user 'joao'@'
    localhost' for table 'aluno'
SELECT * FROM escola.aluno;
```

Em observância ao Princípio do Menor Privilégio, uma conta recém-criada possui privilégios estritamente nulos sobre as tabelas da aplicação até que uma concessão explícita seja homologada.

5. MODELOS CLÁSSICOS DE CONTROLE DE ACESSO

5.1. Controle de Acesso Discricionário (DAC)

No modelo DAC (*Discretionary Access Control*), o proprietário de um recurso concede ou revoga permissões aos demais usuários por meio das cláusulas `GRANT` e `REVOKE` (ELMASRI; NAVATHE, 2011).

```
-- Concessao granular nos niveis de SELECT e INSERT para o usuario Joao
GRANT SELECT, INSERT ON escola.aluno TO 'joao'@'localhost';

-- Verificacao dos privilegios efetivos concedidos a conta
SHOW GRANTS FOR 'joao'@'localhost';

-- Revogacao pontual do privilegio de gravacao
REVOKE INSERT ON escola.aluno FROM 'joao'@'localhost';
```

5.1.1. Granularidade por Coluna (Proteção de Campos Confidenciais)

O SGBD permite conceder acessos restritos a colunas específicas, resguardando atributos sensíveis (como CPF ou salário):

```
CREATE USER 'maria'@'localhost' IDENTIFIED BY 'Maria#2025';

-- Concedendo acesso exclusivamente a colunas operacionais
GRANT SELECT (id_aluno, nome, email) ON escola.aluno TO 'maria'@'localhost';

-- Teste conectado como 'maria':
SELECT nome, email FROM escola.aluno; -- Operacao autorizada

-- Tentativa de acessar a coluna CPF:
-- Retorno do motor: Error 1142 (42000): SELECT command denied on column 'cpf'
SELECT cpf FROM escola.aluno;
```

5.1.2. O Risco da Cláusula WITH GRANT OPTION

A cláusula `WITH GRANT OPTION` permite que o usuário repasse suas próprias permissões a terceiros. Esse poder deve ser monitorado com rigor extremo para evitar a proliferação desgovernada de privilégios administrativos:

```
GRANT SELECT ON escola.aluno TO 'prof_ana'@'localhost' WITH GRANT OPTION;

```

```
-- Revogacao exclusiva do poder de repasse mantendo o privilegio original
REVOKE GRANT OPTION ON escola.aluno FROM 'prof_ana'@'localhost';
```

5.2. Controle de Acesso Obrigatório (MAC)

No modelo MAC (*Mandatory Access Control*), os acessos são regidos por uma política central impositiva. Os dados recebem rótulos de segurança (ex.: 1 – Público, 2 – Confidencial, 3 – Sigiloso) e os usuários recebem autorizações de credenciamento (*clearance*) equivalentes (STALLINGS; BROWN, 2018).

No MySQL Community, essa arquitetura é comumente implementada com visões (*views*) que filtram dinamicamente as tuplas cruzando a identidade da sessão ativa (`USER()`) com a tabela de habilitação:

```
CREATE TABLE documento (
    id INT PRIMARY KEY AUTO_INCREMENT,
    titulo VARCHAR(80) NOT NULL,
    nivel TINYINT NOT NULL CHECK (nivel BETWEEN 1 AND 3)
);

CREATE TABLE habilitacao (
    usuario VARCHAR(30) PRIMARY KEY,
    nivel TINYINT NOT NULL
);

INSERT INTO documento (titulo, nivel) VALUES
('Cardapio Escolar', 1), ('Planilha de Notas', 2), ('Ata Disciplinar Secreta',
    3);

INSERT INTO habilitacao VALUES ('prof_ana', 2), ('prof_bia', 1);

-- Visao com filtro dinamico baseado na identidade da conexao
CREATE VIEW documento_permitido AS
SELECT d.id, d.titulo, d.nivel
FROM documento d
JOIN habilitacao h ON h.usuario = SUBSTRING_INDEX(USER(), '@', 1)
WHERE d.nivel <= h.nivel;
```

Ao liberar privilégios exclusivamente sobre a visão `documento_permitido`, cada usuário visualiza estritamente os documentos compatíveis com o seu nível de credenciamento.

6. CONTROLE DE ACESSO BASEADO EM PAPÉIS (RBAC)

Em médias e grandes corporações, atribuir privilégios diretamente a usuários individuais é inviável e acarreta severos riscos operacionais. O modelo **RBAC** (*Role-Based Access Control*) soluciona essa complexidade associando privilégios a **Papéis** (*Roles*), que representam funções e cargos organizacionais. Os colaboradores recebem um ou mais papéis pertinentes ao seu perfil (SANDHU et al., 1996).

6.1. Ciclo de Vida de Papéis no MySQL

```
-- 1. Criacao dos papeis institucionais
CREATE ROLE 'r_secretaria', 'r_professor', 'r_aluno';

-- 2. Concessao de privilegios direcionados aos papeis
GRANT SELECT, INSERT, UPDATE, DELETE ON escola.aluno TO 'r_secretaria';
GRANT SELECT ON escola.aluno TO 'r_professor';

-- 3. Criacao do colaborador e atribuicao do papel
CREATE USER 'sec_carla'@'localhost' IDENTIFIED BY 'Carla#2025';
GRANT 'r_secretaria' TO 'sec_carla'@'localhost';

-- 4. Ativacao do papel padrao ao conectar
SET DEFAULT ROLE 'r_secretaria' TO 'sec_carla'@'localhost';
```

6.2. Agilidade Operacional na Transferência ou Desligamento

Caso a colaboradora mude de função organizacional, basta revogar o papel anterior sem necessidade de auditar tabelas individuais:

```
-- Inspecao dos privilegios operantes no papel ativo
SHOW GRANTS FOR 'sec_carla'@'localhost' USING 'r_secretaria';

-- Revogacao imediata do papel: todos os privilegios sao cancelados
    instantaneamente
REVOKE 'r_secretaria' FROM 'sec_carla'@'localhost';
```

7. DESAFIO PRÁTICO: MATRIZ DE SEGURANÇA ACADÊMICA

Para consolidar as competências de controle de acesso e segregação funcional, apresentamos o desafio que integra as tabelas `aluno`, `turma`, `matricula` e `nota`.

7.1. Definição da Matriz de Privilégios Corporativa

Papel (Role)	Tabela Alvo	Privilégios Autorizados
`r_secretaria`	`aluno, matricula`	`SELECT, INSERT, UPDATE, DELETE` (Completo)
`r_secretaria`	`turma`	`SELECT, INSERT, UPDATE` (Sem exclusão física)
`r_professor`	`aluno`	`SELECT` (Consulta cadastral)
`r_professor`	`nota`	`SELECT, INSERT, UPDATE` (Lançamento e correção)
`r_aluno`	`nota`	`SELECT` (Consulta de rendimento)

Tabela 4: Matriz de controle de acesso para o subsistema acadêmico.

7.2. Implementação dos Scripts de Concessão

```
CREATE TABLE nota (
    id_aluno INT,
    id_turma INT,
    valor DECIMAL(4,1) CHECK (valor BETWEEN 0 AND 10),
    PRIMARY KEY (id_aluno, id_turma)
);

-- Atribuicao conforme a matriz de seguranca
-- 1. Perfil Secretaria
GRANT SELECT, INSERT, UPDATE, DELETE ON escola.aluno TO 'r_secretaria';
GRANT SELECT, INSERT, UPDATE, DELETE ON escola.matricula TO 'r_secretaria';
GRANT SELECT, INSERT, UPDATE ON escola.turma TO 'r_secretaria';

-- 2. Perfil Docente
GRANT SELECT ON escola.aluno TO 'r_professor';
GRANT SELECT, INSERT, UPDATE ON escola.nota TO 'r_professor';

-- 3. Perfil Discente
GRANT SELECT ON escola.nota TO 'r_aluno';
```

8. SÍNTESE DO CAPÍTULO E COMPARATIVO DE MODELOS

Modelo	Autoridade Decisória	Vantagem Principal	Comandos Típicos
DAC	Proprietário do Objeto	Flexibilidade de compartilhamento	`GRANT, REVOKE`
MAC	Política Central / Rótulo	Alta segurança governamental/militar	Visões, Classificação
RBAC	Função Organizacional	Escalabilidade e facilidade de gestão	`CREATE ROLE, GRANT`

Tabela 5: Comparação funcional entre os modelos de controle de acesso.

REFERÊNCIAS DO CAPÍTULO

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

OWASP. **OWASP Top 10: 2021 - The Ten Most Critical Web Application Security Risks**. Disponível em: https://owasp.org/Top10/. Acesso em: 13 out. 2026.

SALTZER, J. H.; SCHROEDER, M. D. The protection of information in computer systems. **Proceedings of the IEEE**, v. 63, n. 9, p. 1278-1308, 1975.

SANDHU, R. S. et al. Role-based access control models. **IEEE Computer**, v. 29, n. 2, p. 38-47, 1996.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

STALLINGS, W.; BROWN, L. **Segurança de Computadores: Princípios e Práticas**. 4. ed. Rio de Janeiro: GEN LTC, 2018.

EDITORA
OMNIS SCIENTIA

CAPÍTULO 4
CRIPTOGRAFIA, HASH E TÉCNICAS DE PSEUDONIMIZAÇÃO E ANONIMIZAÇÃO EM SGBDs

1. CONTEXTUALIZAÇÃO DO TEMA E A IMPORTÂNCIA DO APRENDIZADO

A segurança lógica e os controles de acesso estudados no Capítulo 3 protegem o banco de dados contra operações ilegítimas intermediadas pelo próprio mecanismo do SGBD. Contudo, em incidentes cibernéticos de alta gravidade, os atacantes frequentemente ultrapassam a camada do software de banco: eles realizam a exfiltração física de arquivos de disco (como as tabelas em formato `.ibd` do InnoDB), obtêm cópias desprotegidas de segurança (*backups*) ou interceptam canais de comunicação desprovidos de proteção (STALLINGS; BROWN, 2018).

Diante de um vazamento físico ou acesso indevido à infraestrutura de armazenamento, as travas de autenticação e os privilégios tornam-se inócuos. É nesse ponto que a **Criptografia em Repouso** (*Encryption at Rest*), as **Funções de Resumo Criptográfico (Hash)** e as **Técnicas de Transformação de Dados** assumem papel decisivo. Dominar essas técnicas é obrigatório para engenheiros de software, administradores de bancos de dados e profissionais de segurança que necessitam resguardar a confidencialidade das informações mesmo diante de brechas de infraestrutura.

1.1. A Problemática Crítica da Proteção Inadequada de Dados

A ausência ou a implementação incorreta de mecanismos criptográficos no SGBD gera vulnerabilidades severas amplamente documentadas no mercado:

- **A Falácia da Proteção Exclusiva por Credenciais:** Se os dados forem armazenados em texto claro (*plaintext*), qualquer administrador de sistemas com acesso ao sistema operacional subjacente, qualquer operador de nuvem ou qualquer invasor que obtenha uma cópia do arquivo de backup terá acesso irrestrito aos segredos corporativos e dados sensíveis dos clientes.

- **O Erro Conceitual entre Criptografia e Hash:** Uma das falhas mais graves na engenharia de software ocorre quando desenvolvedores confundem os propósitos

de cifras simétricas e funções hash. Armazenar senhas com criptografia reversível (em vez de hash unidirecional resistente) permite que um atacante que descubra a chave decifre compulsoriamente todas as credenciais da organização.

- **A Vulnerabilidade de Hashes sem Sal (*Salt*):** O armazenamento de resumos criptográficos simples (como MD5 ou SHA-256 puro) sem adição de sal pseudoaleatório expõe o banco de dados a ataques instantâneos por dicionário e tabelas arco-íris (*rainbow tables*), onde senhas comuns são descobertas em frações de segundo.

- **Exposição Excessiva em Ambientes Analíticos e BI:** É prática recorrente transferir bases de dados completas de produção para ambientes de teste, desenvolvimento e *Business Intelligence* (BI). A exposição de nomes civis, CPFs e prontuários médicos a analistas que necessitam apenas de tendências estatísticas multiplica desnecessariamente a superfície de risco de vazamentos.

- **Corrupção de Dados por Erro de Tipagem:** Um erro técnico clássico no MySQL consiste em armazenar dados cifrados em colunas do tipo `VARCHAR`. Como a saída de um algoritmo de criptografia simétrica consiste em uma sequência de bytes arbitrários, caracteres de escape e bytes nulos corrompem os dados textuais, inviabilizando a decifração posterior da informação original.

O aprendizado deste capítulo instrumentaliza o leitor a selecionar a técnica matemática e arquitetural adequada para cada requisito de negócio, garantindo a proteção dos dados em todo o seu ciclo de vida.

2. FUNDAMENTAÇÃO TEÓRICA: CRIPTOGRAFIA VERSUS FUNÇÃO HASH

Para construir soluções robustas, o engenheiro de dados deve dominar as fronteiras conceituais e operacionais entre criptografia e funções de dispersão (KAUFMAN; PERLMAN; SPECINER, 2002; FERGUSON; SCHNEIER; KOHNO, 2010):

Característica	Criptografia Simétrica (ex.: AES)	Função Hash (ex.: SHA-256)
Direcionalidade	**Bidirecional (Reversível):** Transforma texto claro em texto cifrado e restaura o conteúdo original mediante a chave.	**Unidirecional (Irreversível):** Gera uma impressão digital de tamanho fixo. Não existe operação matemática de volta.
Dependência de Segredo	Exige a custódia rigorosa de uma **chave criptográfica** secreta.	Algoritmo puramente matemático; não depende de chaves (exceto se operado como HMAC).
Finalidade Primária	Proteger a confidencialidade de dados que precisarão ser lidos no futuro (ex.: dados clínicos).	Validação de integridade de dados e verificação de senhas sem necessidade de conhecê-las.
Efeito Avalanche	Presente por blocos de modo de operação.	Estrito: a alteração de um único bit na entrada altera cerca de 50% dos bits da saída.

Tabela 6: Diferenças estruturais entre criptografia simétrica e funções hash.

2.1. O Uso Mandatório de Resumo Criptográfico com Sal (Salt)

No armazenamento seguro de senhas, a adição de uma cadeia pseudoaleatória exclusiva (*salt*) a cada entrada inviabiliza que invasores utilizem bancos de dados de hashes pré-computados:

```
-- Demonstracao conceitual de hash salgado
SET @sal = 'x#9K$qL@2026';
SELECT SHA2(CONCAT('SenhaDoUsuario', @sal), 256) AS hash_blindado;
```

3. IMPLEMENTAÇÃO DE CRIPTOGRAFIA SIMÉTRICA COM AES NO MYSQL

O padrão **AES** (*Advanced Encryption Standard*), formalizado pelo NIST através do FIPS 197 (NIST, 2001), é a cifra de bloco simétrica de referência adotada internacionalmente por corporações e governos. No MySQL, o algoritmo opera de modo nativo através das funções `AES_ENCRYPT()` e `AES_DECRYPT()`.

3.1. Requisitos de Tipagem: O Uso Exclusivo de VARBINARY

Colunas destinadas ao armazenamento de saídas criptográficas devem ser declaradas estritamente com os tipos binários **`VARBINARY`** ou **`BLOB`**. A utilização de tipos textuais (`VARCHAR`) acarreta corrupção irreversível da cifra devido ao tratamento de tabelas de caracteres (*charsets*) e caracteres de controle (ORACLE, 2024).

3.2. Modelagem e Inserção de Dados Sensíveis

Considere uma relação cadastral contendo informações clínicas em que os diagnósticos representam dados de altíssima sensibilidade:

```
USE escola;

CREATE TABLE titular (
    id_titular INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    email VARCHAR(100) NOT NULL,
    cidade VARCHAR(40),
    saude VARBINARY(255), -- Tipagem binaria mandatoria para cifra AES
    consentimento TINYINT(1) DEFAULT 0
);
```

3.3. Ciclo de Criptografia e Consulta Condicional

Em ambientes de produção, a chave criptográfica é gerida em cofres de chaves (*Key Management Services* – KMS). No laboratório a seguir, derivamos uma chave de sessão de 256 bits:

```
-- Derivacao de chave a partir de um segredo mestre
SET @k = SHA2('chave-mestra-unirv-2026', 256);
```

```
3
4  -- Insercao com criptografia do dado de saude
5  INSERT INTO titular (nome, cpf, email, cidade, saude, consentimento) VALUES
6  ('Ana Paula', '11111111111', 'ana@mail.com', 'Rio Verde', AES_ENCRYPT('asma
       cronica', @k), 1),
7  ('Beatriz Costa', '22222222222', 'bia@mail.com', 'Jatai', AES_ENCRYPT('rinite
       alergica', @k), 0);
```

3.3.1. Consulta Desprovida da Chave Criptográfica

Caso a tabela seja consultada sem o fornecimento da chave:

```
1  SELECT nome, saude FROM titular;
```

O SGBD exibe apenas sequências binárias inteligíveis exclusivamente como bytes brutos (ex.: `0x7B28A94F1C...`). O dado médico permanece inviolado.

3.3.2. Consulta Autorizada com a Chave e Coerção de Tipo

A aplicação devidamente autorizada que detém a posse da chave decifra e converte os bytes em cadeia legível com `CAST(... AS CHAR)`:

```
1  SELECT nome,
2         CAST(AES_DECRYPT(saude, @k) AS CHAR) AS diagnostico_saude
3  FROM titular;
```

4. TÉCNICAS DE PSEUDONIMIZAÇÃO EM ENGENHARIA DE DADOS

A **Pseudonimização** é o tratamento de dados pessoais por meio do qual um registro perde a possibilidade de associação direta ou indireta a um indivíduo específico, senão pelo uso de informação adicional mantida separadamente pelo controlador em ambiente controlado e isolado (BRASIL, 2018; ANPD, 2023).

Trata-se de um processo **reversível**, porém essa reversibilidade fica estritamente restrita a quem possui a custódia formal da chave de correlação ou tabela de mapeamento.

4.1. Arquitetura com Tabela de Mapeamento Separada

```sql
CREATE TABLE mapa_pseudonimo (
    codigo CHAR(10) PRIMARY KEY,
    id_titular INT UNIQUE NOT NULL,
    FOREIGN KEY (id_titular) REFERENCES titular(id_titular)
    ON DELETE CASCADE
);
```

4.2. Geração Algorítmica dos Códigos Pseudonimizados

Os identificadores artificiais são gerados aplicando resumo criptográfico sobre o CPF concatenado a um segredo de sal institucional:

```sql
SET @sal = '#segredo-institucional-2026';

INSERT INTO mapa_pseudonimo (codigo, id_titular)
SELECT LEFT(SHA2(CONCAT(cpf, @sal), 256), 10), id_titular
FROM titular;

SELECT * FROM mapa_pseudonimo;
```

4.3. Criação de Visão Analítica para Inteligência de Negócios (BI)

Equipes de cientistas de dados e analistas de BI necessitam realizar correlações demográficas sem conhecer a identidade civil dos indivíduos:

```sql
CREATE VIEW analise_pseudonimizada AS
SELECT m.codigo, t.cidade
FROM titular t
```

```
JOIN mapa_pseudonimo m USING (id_titular);

SELECT * FROM analise_pseudonimizada;
```

O relatório expõe apenas códigos abstratos e a cidade, eliminando o risco de exposição de nomes e documentos civis em relatórios gerenciais.

5. ANONIMIZAÇÃO DEFINITIVA E MASCARAMENTO DINÂMICO

5.1. Anonimização: A Transformação Irreversível

A **Anonimização** ocorre quando um dado pessoal é submetido a procedimentos técnicos em decorrência dos quais o titular não pode mais ser identificado por nenhum meio razoável disponível (ANPD, 2023). Ao contrário da pseudonimização, a anonimização é um processo **definitivo e sem volta**.

Em bancos relacionais, a anonimização é realizada principalmente por meio de transformações agregadas que descartam identificadores individuais e mapas relacionais:

```sql
-- Geracao de relatorio inteiramente anonimizado
CREATE TABLE relatorio_anonimo AS
SELECT cidade, COUNT(*) AS total_titulares
FROM titular
GROUP BY cidade;

SELECT * FROM relatorio_anonimo;
```

O resultado reflete apenas quantitativos geográficos agregados. É matematicamente impossível reverter a contagem para identificar os indivíduos que compõem o grupo.

5.2. Mascaramento Dinâmico de Dados (Data Masking)

O mascaramento dinâmico consiste na exibição ofuscada de campos confidenciais em interfaces operacionais e centrais de atendimento (SAC), em estrita observância ao **Princípio da Minimização** (necessidade estrita de visualização para o atendimento).

```sql
CREATE VIEW titular_mascarado AS
SELECT nome,
       CONCAT(LEFT(email, 2), '***@', SUBSTRING_INDEX(email, '@', -1)) AS email
           ,
       CONCAT('*******', RIGHT(cpf, 4)) AS cpf
FROM titular;

SELECT * FROM titular_mascarado;
```

Resultado Gerado para a Interface de Atendimento:

```
+---------------+----------------+-------------+
| nome          | email          | cpf         |
+---------------+----------------+-------------+
```

```
| Ana Paula     | an***@mail.com | *******1111 |
| Beatriz Costa | bi***@mail.com | *******2222 |
+---------------+----------------+-------------+
```

O atendente visualiza os dados suficientes para conferência do titular no atendimento telefônico, sem expor o e-mail completo ou o CPF a olhares indiscretos ou captura indevida.

6. SÍNTESE DO CAPÍTULO E QUADRO COMPARATIVO

Técnica	Reversibilidade	Mecanismo SQL Típico	Caso de Uso Primário
Hash	Não Reversível	`SHA2(CONCAT(dado, sal), 256)`	Senhas, integridade de
Criptografia	Reversível (com chave)	`AES_ENCRYPT, AES_DECRYPT`	Dados sensíveis em rep
Pseudonimização	Reversível (com mapa)	`Tabela satélite + Código`	Bases analíticas e pesqu
Anonimização	Irreversível	`GROUP BY, COUNT(*)`	Estatísticas e dados abe
Mascaramento	Visual / Parcial	`CONCAT, LEFT, SUBSTRING`	Telas de SAC e atendim

Tabela 7: Quadro comparativo de técnicas de preservação de dados.

REFERÊNCIAS DO CAPÍTULO

ANPD. AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. **Guia Orientativo: Tratamento de Dados Pessoais para Fins Acadêmicos e Estudos por Órgão de Pesquisa**. Brasília: ANPD, 2023.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018.

FERGUSON, N.; SCHNEIER, B.; KOHNO, T. **Cryptography Engineering: Design Principles and Practical Applications**. Indianapolis: Wiley Publishing, 2010.

KAUFMAN, C.; PERLMAN, R.; SPECINER, M. **Network Security: Private Communication in a Public World**. 2. ed. Upper Saddle River: Prentice Hall, 2002.

NIST. NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **FIPS PUB 197: Advanced Encryption Standard (AES)**. Gaithersburg: U.S. Department of Commerce, 2001.

ORACLE. **MySQL 8.0 Reference Manual: Encryption and Compression Functions**. Redwood City: Oracle Corporation, 2024. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 13 out. 2026.

STALLINGS, W.; BROWN, L. **Segurança de Computadores: Princípios e Práticas**. 4. ed. Rio de Janeiro: GEN LTC, 2018.

CAPÍTULO 5
GOVERNANÇA DE DADOS E CONFORMIDADE COM A LGPD NA PRÁTICA COM SQL

1. CONTEXTUALIZAÇÃO DO TEMA E A IMPORTÂNCIA DO APRENDIZADO

A consolidação da sociedade da informação e a transformação digital de serviços públicos e privados elevaram os dados pessoais à categoria de direitos fundamentais da personalidade. No Brasil, esse paradigma foi formalizado pela Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais – LGPD), e fortalecido pela Emenda Constitucional nº 115/2022, que incluiu a proteção de dados pessoais no rol de garantias fundamentais da Constituição Federal (BRASIL, 2018, 2022).

Para os profissionais que projetam, desenvolvem e sustentam sistemas computacionais, a conformidade regulatória deixou de ser um tópico restrito aos departamentos jurídicos e tornou-se um **requisito arquitetural compulsório de engenharia de software**. Compreender a governança de dados no nível do SGBD é hoje um dos maiores diferenciais competitivos e uma das exigências mais prementes do mercado de tecnologia.

1.1. A Problemática Crítica da Não Conformidade e Riscos Reais

A negligência na implementação de mecanismos de privacidade e governança em bancos de dados expõe as organizações a consequências financeiras, operacionais e de reputação sem precedentes:

- **Penalidades Financeiras Severas (Art. 52 da LGPD):** O descumprimento das normas sujeita a instituição a multas administrativas aplicadas pela Autoridade Nacional de Proteção de Dados (ANPD) de até **2% do faturamento da empresa**, limitadas a **R$ 50.000.000,00 (cinquenta milhões de reais) por infração**, além de multas diárias progressivas.
- **Interrupção e Bloqueio Operacional:** A ANPD possui competência legal para determinar o bloqueio temporário ou a eliminação compulsória do banco de dados ao qual se refere a infração. Para empresas dependentes de plataformas digitais, tal sanção equivale à paralisação total das atividades empresariais.

- **O Abismo entre Políticas Documentais e Prática do SGBD:** Um dos erros mais graves observados no mercado consiste em elaborar termos de consentimento e políticas de privacidade formais em papel, enquanto as tabelas do banco continuam sem inventário de finalidade, sem segregação de acessos e sem trilhas de auditoria. Auditorias regulatórias exigem evidências técnicas diretas extraídas do banco de dados, e não apenas declarações de intenção.

- **Incapacidade Técnica de Atender aos Direitos dos Titulares:** O artigo 18 da LGPD assegura que qualquer titular pode requerer confirmação de tratamento, acesso, correção, portabilidade estruturada e eliminação de seus dados pessoais. Se o banco não foi projetado com chaves rastreáveis e rotinas operacionais preparadas, o tempo de atendimento extrapola os prazos legais, gerando denúncias e litígios judiciais indenizatórios.

- **Ausência de Prestação de Contas (*Accountability*):** Quando um registro é indevidamente alterado ou subtraído, a ausência de trilhas automáticas de auditoria impede a identificação do agente responsável, transferindo a presunção de culpa e negligência diretamente para os gestores da organização.

O aprendizado deste capítulo capacita o leitor a traduzir os princípios e mandamentos da legislação de privacidade em esquemas relacionais, gatilhos de auditoria invioláveis e comandos SQL que operacionalizam diretamente os direitos dos cidadãos.

2. FUNDAMENTAÇÃO TEÓRICA E JURÍDICO-TECNOLÓGICA DA LGPD

A proteção de dados pessoais funda-se no princípio da **Autodeterminação Informativa**, que confere ao indivíduo o controle sobre a coleta, utilização e circulação dos seus dados pessoais (BIONI, 2021; DONEDA, 2020).

A LGPD (BRASIL, 2018) estabelece definições rigorosas para os atores e categorias envolvidos no tratamento:

- **Titular:** A pessoa natural (física) a quem se referem os dados pessoais que são objeto de tratamento.
- **Controlador:** Pessoa física ou jurídica a quem competem as decisões fundamentais sobre as finalidades e meios de tratamento dos dados.
- **Operador:** Pessoa física ou jurídica que realiza o tratamento de dados pessoais em nome e sob as ordens do controlador.
- **Encarregado pelo Tratamento (DPO – *Data Protection Officer*):** Indivíduo indicado para atuar como canal de comunicação entre o controlador, os titulares e a autoridade nacional.
- **Dado Pessoal:** Informação relacionada a pessoa natural identificada ou identificável (art. 5º, I).
- **Dado Pessoal Sensível:** Dado pessoal sobre origem racial ou étnica, convicção religiosa, opinião política, filiação a sindicato, dado referente à saúde ou à vida sexual, dado genético ou biométrico (art. 5º, II). Essa categoria requer bases legais mais restritivas e medidas de segurança reforçadas.

2.1. Princípios Basilares Aplicados à Engenharia de Dados

Dentre os princípios esculpidos no art. 6º da legislação, destacam-se para a arquitetura de banco de dados:

1. **Princípio da Finalidade:** O tratamento deve ter propósitos legítimos e específicos informados ao titular, vedado o uso posterior divergente.

2. **Princípio da Necessidade (Minimização):** Coleta limitada ao mínimo estritamente indispensável para a realização da finalidade. Tabelas não devem possuir colunas supérfluas.

3. **Princípio da Segurança e Prevenção:** Emprego de medidas técnicas aptas a resguardar os dados de acessos não autorizados.

4. **Princípio da Responsabilização e Prestação de Contas (*Accountability*):** Demonstração pelo controlador de medidas eficazes capazes de comprovar documental e tecnicamente o cumprimento da lei (PINHEIRO, 2020).

3. INVENTÁRIO DE DADOS E REGISTRO DE FINALIDADE EM SQL

O alicerce da governança de dados corporativa consiste na catalogação e no inventário formal de dados (*Data Mapping* ou RoPA – *Record of Processing Activities*).

No SGBD, esse inventário pode ser materializado em uma relação de metadados de governança, associando cada atributo da base à sua categoria jurídica e à respectiva finalidade legal documentada:

```sql
USE escola;

-- Tabela de governanca de dados do esquema relacional
CREATE TABLE inventario_dados (
    coluna VARCHAR(30) PRIMARY KEY,
    tipo VARCHAR(10) NOT NULL CHECK (tipo IN ('PESSOAL', 'SENSIVEL')),
    finalidade VARCHAR(60) NOT NULL
);

-- Catalogacao das colunas e de seus propositos declarados
INSERT INTO inventario_dados VALUES
('nome', 'PESSOAL', 'Identificacao civil do titular'),
('cpf', 'PESSOAL', 'Obrigacao legal perante orgaos governamentais'),
('email', 'PESSOAL', 'Canal oficial de comunicacao institucional'),
('cidade', 'PESSOAL', 'Analise demografica e regional'),
('saude', 'SENSIVEL', 'Adequacao medica e atendimento emergencial');

SELECT * FROM inventario_dados;
```

Com esse esquema, o banco de dados auto-documenta as finalidades de cada atributo, assegurando conformidade com o princípio da transparência perante auditorias da ANPD.

4. TRILHAS DE AUDITORIA CONTÍNUA COM GATILHOS (TRIGGERS)

Para satisfazer o princípio da prestação de contas (*accountability*), a organização deve comprovar a rastreabilidade temporal de modificações em dados pessoais, registrando o operador, o evento e o instante da mutação (ANPD, 2021).

4.1. Modelagem da Tabela de Auditoria

A tabela de auditoria deve operar sob o paradigma de inserção exclusiva (*append-only*), sem permissão de alteração ou exclusão para usuários comuns:

```
CREATE TABLE log_auditoria (
    id INT PRIMARY KEY AUTO_INCREMENT,
    usuario VARCHAR(100) NOT NULL,
    acao VARCHAR(10) NOT NULL,
    id_titular INT,
    data_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

4.2. Implementação do Gatilho AFTER UPDATE

Utilizamos um gatilho configurado para o momento `AFTER UPDATE`, garantindo que a gravação do log ocorra apenas se a atualização na tabela mestre de titulares for consumada com êxito:

```
DELIMITER $$

CREATE TRIGGER audit_titular_upd
AFTER UPDATE ON titular
FOR EACH ROW
BEGIN
    INSERT INTO log_auditoria (usuario, acao, id_titular)
    VALUES (CURRENT_USER(), 'UPDATE', NEW.id_titular);
END$$

DELIMITER ;
```

4.3. Teste Operacional de Auditoria

```
-- Simulacao de atualizacao cadastral por um operador
UPDATE titular
SET email = 'ana.souza@mail.com'
WHERE id_titular = 1;

-- Inspecao imediata da trilha de auditoria
SELECT id, usuario, acao, id_titular, data_hora
FROM log_auditoria;
```

Resultado Retornado pelo SGBD:

```
+----+----------------+--------+------------+---------------------+
| id | usuario        | acao   | id_titular | data_hora           |
+----+----------------+--------+------------+---------------------+
|  1 | root@localhost | UPDATE |          1 | 2026-10-13 10:45:00 |
+----+----------------+--------+------------+---------------------+
```

Nota Técnica sobre Consultas (SELECT): Em SQL, gatilhos padrão disparam apenas mediante instruções DML de alteração (`INSERT`, `UPDATE`, `DELETE`). O monitoramento de consultas `SELECT` envolvendo dados altamente confidenciais deve ser viabilizado por meio do plugin de auditoria do SGBD (como o *MySQL Enterprise Audit*) ou por logs de aplicação integrados ao servidor web.

5. ATENDIMENTO TEMPESTIVO AOS DIREITOS DOS TITULARES

O art. 18 da LGPD estabelece um catálogo de direitos fundamentais que o titular pode exercer a qualquer momento mediante requisição formal dirigida ao controlador. A seguir, traduzimos os cinco principais direitos do titular em operações transacionais diretas em SQL.

5.1. Direito 1: Confirmação da Existência de Tratamento

O titular tem o direito de indagar se a instituição possui qualquer dado a seu respeito na base:

```sql
-- Verificacao rapida e otimizada por indice do CPF
SELECT COUNT(*) AS existe_tratamento
FROM titular
WHERE cpf = '11111111111';
```

Se o retorno for superior a zero, o tratamento ativo é formalmente confirmado.

5.2. Direito 2: Acesso aos Dados Cadastrais

Confirmada a existência, o titular pode exigir a exibição de todas as suas informações custodiadas:

```sql
SELECT nome, cpf, email, cidade
FROM titular
WHERE cpf = '11111111111';
```

5.3. Direito 3: Correção de Dados Incompletos, Inexatos ou Desatualizados

O titular tem direito à retificação tempestiva de dados cadastrais incorretos:

```sql
UPDATE titular
SET cidade = 'Jatai'
WHERE cpf = '11111111111';
```

5.4. Direito 4: Portabilidade dos Dados a Outro Fornecedor de Serviço

O titular pode solicitar a entrega de seus dados em formato estruturado e interoperável de leitura mecânica. No MySQL 8.0+, isso é atendido de forma imediata e elegante por meio da função nativa de geração JSON:

```
SELECT JSON_OBJECT(
    'nome', nome,
    'cpf', cpf,
    'email', email,
    'cidade', cidade
) AS pacote_portabilidade_json
FROM titular
WHERE cpf = '11111111111';
```

O resultado é um objeto JSON padronizado e pronto para integração com qualquer outro sistema ou plataforma externa.

5.5. Direito 5: Eliminação dos Dados Pessoais (Direito ao Esquecimento)

Ao término da finalidade ou mediante revogação formal de consentimento, o titular pode exigir a exclusão de seus dados pessoais armazenados. Graças à integridade referencial configurada com `ON DELETE CASCADE` no Capítulo 4, a exclusão na tabela mestre remove automaticamente todos os registros associados em tabelas satélites:

```
-- Exclusao do registro do titular com CPF '22222222222'
DELETE FROM titular
WHERE cpf = '22222222222';

-- Conferencia da limpeza automatica do mapa de pseudonimos
SELECT * FROM mapa_pseudonimo WHERE id_titular = 2; -- Retorna zero registros!
SELECT * FROM titular WHERE cpf = '22222222222';     -- Registro eliminado!
```

6. DESAFIO PRÁTICO FINAL E ESTUDO DE CASO CORPORATIVO

Para consolidar as competências de governança de dados e privacidade, apresentamos o estudo de caso de modelagem e mitigação de riscos.

6.1. Contexto do Problema

Uma instituição acadêmica e plataforma de comércio eletrônico mantém cadastros de alunos, faturas financeiras e registros médicos de acessibilidade. O conselho diretor exige a adequação imediata aos preceitos da LGPD.

Tarefas Solicitadas ao Aluno / DBA:

1. Identificar e classificar os atributos entre dados pessoais e dados pessoais sensíveis.
2. Mapear os agentes formais de tratamento no cenário institucional.
3. Implementar uma medida técnica preventiva em SQL que garanta a privacidade dos titulares em consultas operacionais de suporte.

6.2. Resolução Comentada do Estudo de Caso

1. Classificação dos Dados Institucionais

- **Dados Pessoais:** Nome completo, CPF, e-mail institucional, endereço residencial e histórico de compras/notas.
- **Dados Pessoais Sensíveis:** Informações de saúde, laudos médicos e condições especiais (exigindo criptografia AES e isolamento em tabelas com restrição de acesso estrita).

2. Mapeamento de Agentes de Tratamento

- **Titular:** Aluno, candidato ao vestibular ou cliente da loja virtual.
- **Controlador:** A instituição mantenedora que define as diretrizes do negócio.
- **Operador:** A empresa de hospedagem em nuvem e a processadora terceirizada de pagamentos.

- **Encarregado (DPO):** Profissional designado formalmente no portal de transparência da instituição.

3. Medida Técnica Implementada em SQL

Para viabilizar que atendentes de suporte confirmem dados dos alunos sem expor desnecessariamente informações integrais na tela, constrói-se uma visão com mascaramento dinâmico:

```sql
CREATE VIEW atendimento_seguro_aluno AS
SELECT id_titular,
       nome,
       CONCAT(LEFT(email, 2), '***@', SUBSTRING_INDEX(email, '@', -1)) AS
           email_protegido,
       CONCAT('***.***.', SUBSTRING(cpf, 7, 3), '-', RIGHT(cpf, 2)) AS
           cpf_mascarado,
       cidade
FROM titular;

SELECT * FROM atendimento_seguro_aluno;
```

7. CONSIDERAÇÕES FINAIS DA OBRA

Ao longo deste livro técnico, percorremos a espinha dorsal da engenharia moderna de bancos de dados relacionais:

- A **Integridade Declarativa** (Domínio e Entidade) que impede a corrupção estrutural da base;
- A **Integridade Referencial e Procedural** (Chaves Estrangeiras e Gatilhos) que preserva a coerência entre relacionamentos e valida regras dinâmicas de negócio;
- A **Segurança e Controle de Acesso** (DAC, MAC e RBAC) que blinda os dados contra invasões e materializa o princípio do menor privilégio;
- A **Criptografia, Hash e Anonimização** que resguardam ativos críticos mesmo diante de incidentes físicos ou vazamentos acidentais;
- A **Governança e Conformidade com a LGPD** que harmoniza os esquemas de dados com a legislação civil e assegura respeito integral aos direitos dos titulares.

Projetar bancos de dados íntegros, seguros e em conformidade não é um evento isolado, mas uma disciplina contínua de engenharia. Que este guia sirva de referência sólida para a sua prática profissional e acadêmica!

REFERÊNCIAS DO CAPÍTULO

ANPD. AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. **Guia Orientativo: Segurança da Informação para Agentes de Tratamento de Pequeno Porte**. Brasília: ANPD, 2021.

BIONI, R. C. **Proteção de Dados Pessoais: A Função e os Limites do Consentimento**. 3. ed. Rio de Janeiro: Forense, 2021.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018.

BRASIL. **Emenda Constitucional nº 115, de 10 de fevereiro de 2022**. Altera a Constituição Federal para incluir a proteção de dados pessoais entre os direitos e garantias fundamentais. Diário Oficial da União: Brasília, DF, 11 fev. 2022.

DONEDA, D. **Da Privacidade à Proteção de Dados Pessoais: Elementos da Formação do seu Conceito**. 2. ed. São Paulo: Thomson Reuters Brasil, 2020.

PINHEIRO, P. P. **Proteção de Dados Pessoais: Comentários à Lei nº 13.709/2018 (LGPD)**. 2. ed. São Paulo: Saraiva Educação, 2020.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

EDITORA OMNIS SCIENTIA

REFERÊNCIAS GERAIS DA OBRA

ANPD. AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. **Guia Orientativo: Segurança da Informação para Agentes de Tratamento de Pequeno Porte**. Brasília: ANPD, 2021.

ANPD. AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. **Guia Orientativo: Tratamento de Dados Pessoais para Fins Acadêmicos e Estudos por Órgão de Pesquisa**. Brasília: ANPD, 2023.

BIONI, R. C. **Proteção de Dados Pessoais: A Função e os Limites do Consentimento**. 3. ed. Rio de Janeiro: Forense, 2021.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018.

BRASIL. **Emenda Constitucional nº 115, de 10 de fevereiro de 2022**. Altera a Constituição Federal para incluir a proteção de dados pessoais entre os direitos e garantias fundamentais. Diário Oficial da União: Brasília, DF, 11 fev. 2022.

CODD, E. F. A relational model of data for large shared data banks. **Communications of the ACM**, v. 13, n. 6, p. 377-387, 1970.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.

DONEDA, D. **Da Privacidade à Proteção de Dados Pessoais: Elementos da Formação do seu Conceito**. 2. ed. São Paulo: Thomson Reuters Brasil, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

FERGUSON, N.; SCHNEIER, B.; KOHNO, T. **Cryptography Engineering: Design Principles and Practical Applications**. Indianapolis: Wiley Publishing, 2010.

KAUFMAN, C.; PERLMAN, R.; SPECINER, M. **Network Security: Private Communication in a Public World**. 2. ed. Upper Saddle River: Prentice Hall, 2002.

MELTON, J.; SIMON, A. R. **SQL: 1999 - Understanding Relational Language Components**. San Francisco: Morgan Kaufmann, 2002.

NIST. NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **FIPS PUB 197: Advanced Encryption Standard (AES)**. Gaithersburg: U.S. Department of Commerce, 2001.

ORACLE. **MySQL 8.0 Reference Manual**. Redwood City: Oracle Corporation, 2024. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 13 out. 2026.

OWASP. **OWASP Top 10: 2021 - The Ten Most Critical Web Application Security Risks**. Open Web Application Security Project, 2021. Disponível em: https://owasp.org/Top10/. Acesso em: 13 out. 2026.

PINHEIRO, P. P. **Proteção de Dados Pessoais: Comentários à Lei nº 13.709/2018 (LGPD)**. 2. ed. São Paulo: Saraiva Educação, 2020.

SALTZER, J. H.; SCHROEDER, M. D. The protection of information in computer systems. **Proceedings of the IEEE**, v. 63, n. 9, p. 1278-1308, 1975.

SANDHU, R. S. et al. Role-based access control models. **IEEE Computer**, v. 29, n. 2, p. 38-47, 1996.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

STALLINGS, W.; BROWN, L. **Segurança de Computadores: Princípios e Práticas**. 4. ed. Rio de Janeiro: GEN LTC, 2018.